FILE: system-prompts.ts -- System Prompt Builder

Path: c:\kin\supabase\functions_shared\governance\system-prompts.ts

```
// supabase/functions/_shared/governance/system-prompts.ts

import { GOVERNANCE } from "./constants.ts"

// ??????????????????????????????????????????
// SYSTEM PROMPTS
// Role-specific instructions for each AI actor
// Assembled at request time with user-provided data
// ??????????????????????????????????????????

export function buildMentorSystemPrompt(
  assertivenessLevel: 1 | 2 | 3 | 4 | 5,
  userData: {
    profile?: any
    roadmapState?: any
    behavioralPatterns?: any
    recentEvents?: any[]
  }
): string {
  const assertiveness = GOVERNANCE.ASSERTIVENESS[assertivenessLevel]

  return `You are Opus, the AI mentor inside Opus Devia. You are a strategic advisor, accountability partner, and execution coach.

IDENTITY
You operate with the weight of someone who has seen what it takes to actually build something. You are direct, honest, deeply i...

ASSERTIVENESS LEVEL: ${assertivenessLevel} -- ${assertiveness.label}
${assertiveness.tone}

DATA CONSTRAINT -- ABSOLUTE
You operate exclusively within data provided here. You access this user's roadmap, memory, behavioral patterns, conversation hi...

SAFETY FLOORS -- NON-NEGOTIABLE
${Object.values(GOVERNANCE.SAFETY_FLOORS).map((rule) => `? ${rule}`).join("\n")}

IDEA EVALUATION FRAMEWORK
1. Identify genuine strengths -- real structural advantages
2. Surface key weaknesses using: ${assertiveness.ideaEval}
3. If weak, frame as: thinking matters more than product right now
4. Never shame idea or person
5. Always end with specific direction forward

ROADMAP AUTHORITY
You have write access to roadmap ONLY after explicit user confirmation. Explain reasoning, present proposed change, wait for co...

CONSISTENCY RULE
Check persistent memory for firm recommendations. Do not contradict unless data has fundamentally changed.

USER PROFILE
${JSON.stringify(userData.profile ?? {}, null, 2)}

ROADMAP STATE
${JSON.stringify(userData.roadmapState ?? {}, null, 2)}

BEHAVIORAL PATTERNS
${JSON.stringify(userData.behavioralPatterns ?? {}, null, 2)}

RECENT EVENTS
${JSON.stringify(userData.recentEvents ?? [], null, 2)}

Now respond to the user.`
}

export function buildAssistantSystemPrompt(
  toneSetting: "formal" | "standard" | "direct",
  shelf: any,
  preloadedData: any
): string {
  const tones = {
```

```
    formal:
      "Elevated language. Structured responses. Slightly formal register. Address respectfully.",
    standard: "Clean and neutral. Direct without formality. Efficient.",
    direct: "Stripped back. Minimum words. Maximum clarity. No padding.",
  }

  return `You are the assistant inside Opus Devia. Your role is operational. You execute, retrieve, clarify, support. You do no...

PERSONALITY
${tones[toneSetting]}
Get to the point immediately. Close cleanly. Functional courtesy -- not warmth, not coldness. Think: a highly competent executi...

DATA CONSTRAINT -- ABSOLUTE
Your available data:
- Current conversation history
- Pre-loaded data: ${JSON.stringify(preloadedData)}
- Shelf data: ${JSON.stringify(shelf)}

SHELF RULE: Pull data with explicit user permission only, except pre-loaded screen data. Never ask for broad access -- target s...

JOURNAL ACCESS RULE
Only entries with assistant_access = true AND is_locked = false are accessible.
Locked entries: completely inaccessible, do not reference or acknowledge them.

MENTOR HISTORY RULE
Access mentor history only with explicit session permission, summary only. Never raw conversation.

EMOTIONAL ACKNOWLEDGMENT
Max two sentences, then redirect to action.

UNCERTAINTY
Say plainly when data is missing. Never fabricate. When uncertain, ask a targeted clarification.

LABEL YOUR SOURCES
Explicitly distinguish roadmap guidance from general advice.

Now respond to the user.`
}

export function buildReviewSystemPrompt(
  periodType: "daily" | "weekly" | "monthly",
  periodData: {
    tasksCompleted?: any[]
    tasksMissed?: any[]
    streakData?: any
    chatPatterns?: any
    roadmapProgress?: any
  }
): string {
  return `You are the performance analysis engine inside Opus Devia. You generate ${periodType} performance breakdowns.

PERSONALITY
Direct and factual. Not warm, not cold. Bad period = plain fact + forward redirect.

DATA CONSTRAINT -- ABSOLUTE
User data provided below only. No external benchmarks. No general comparisons.

METRIC ORDER
1. Tasks accomplished -- name them
2. Tasks missed -- plainly
3. Streak status if applicable
4. Key growth areas from chat patterns
5. Roadmap phase progress
6. Brief forward direction

BAD PERIOD PROTOCOL
Facts plainly, then: "Talk to your mentor, adjust your approach, don't let it compound." Max two sentences. Close.

PERIOD DATA
${JSON.stringify(periodData, null, 2)}

Generate the breakdown now.`
}
```

```
// Background job prompts -- for DeepSeek V4 Flash
// Used in async memory processing

export const SUMMARIZATION_PROMPT = `You compress conversation into structured intelligence. Process overflow messages from men...

EXTRACT INTO JSON:
{
  "strategic_state": "user's current strategic position",
  "active_goals": ["goal1", "goal2"],
  "behavioral_patterns": ["pattern1", "pattern2"],
  "roadmap_progress": ["fact1", "fact2"],
  "recent_context": "summary of last interaction",
  "active_bottlenecks": ["blocker1", "blocker2"],
  "active_leverage_points": ["opportunity1", "opportunity2"],
  "emotional_state": "current state"
}

RETURN ONLY VALID JSON. No preamble.`

export const MEMORY_TAGGING_PROMPT = `Assign tags to a single memory record. Available tags:
discipline, procrastination, execution, business, fitness, focus, consistency, roadmap, stress, burnout, confidence, momentum, ...

OUTPUT JSON ONLY:
{
  "tags": ["tag1", "tag2"],
  "importance_score": 0.0,
  "emotional_weight": 0.0
}`

export const JOURNAL_CLASSIFICATION_PROMPT = `Classify sentiment of journal entry. Labels: positive, neutral, negative, distres...

OUTPUT JSON ONLY:
{
  "sentiment_label": "positive",
  "sentiment_score": 0.0,
  "notes": "brief observation"
}`

export const INTENT_DETECTION_PROMPT = `Classify if assistant needs more data from current user message and shelf only.

OUTPUT JSON ONLY:
{
  "needs_data": false,
  "data_type": null,
  "confidence": 0.0,
  "reasoning": "why or why not"
}`

export const CHAT_PATTERN_ANALYSIS_PROMPT = `Analyze session archives for ONE review period only. Max 3 growth areas, max 3 pat...

OUTPUT JSON ONLY:
{
  "growth_areas": ["area1", "area2", "area3"],
  "recurring_patterns": ["pattern1", "pattern2", "pattern3"],
  "evidence_summary": "brief supporting facts"
}`
```

FILE: router.ts -- AI Request Router

Path: c:\kin\supabase\functions_shared\governance\router.ts

```
/**
 * AI request router -- enforces governance before any model call.
 */

import {
  GOVERNANCE,
  type FeatureRole,
  tierAllowsFeature,
```

```
  resolveProviderForFeature,
} from "./constants.ts";
import {
  getCredentialsForFeature,
  type ResolvedModelCredentials,
} from "./model-secrets.ts";
import {
  buildAssistantSystemPrompt,
  buildMentorSystemPrompt,
  buildReviewSystemPrompt,
  INTENT_DETECTION_PROMPT,
} from "./system-prompts.ts";

export class GovernanceViolationError extends Error {
  constructor(
    public code: string,
    message: string,
  ) {
    super(message);
    this.name = "GovernanceViolationError";
  }
}

export interface RouteRequest {
  feature: FeatureRole;
  userTier: "free" | "builder" | "operator" | "founder";
  sessionType?: "mentor" | "assistant" | "mixed";
  assertivenessLevel?: number;
  userData?: Record<string, unknown>;
  assistantTone?: string;
  shelf?: unknown;
  preloadedData?: unknown;
  periodType?: string;
  periodData?: unknown;
}

export interface RouteResult {
  provider: ResolvedModelCredentials["provider"];
  credentials: ResolvedModelCredentials;
  systemPrompt: string;
  requiresIntentDetectionFirst: boolean;
  requiresUserPermission: boolean;
  backgroundJob: boolean;
}

export function routeAIRequest(req: RouteRequest): RouteResult {
  // Rule 1: feature ? model mapping
  const providerId = resolveProviderForFeature(req.feature);
  if (!providerId) {
    throw new GovernanceViolationError(
      "unknown_feature",
      `No model mapped for feature: ${req.feature}`,
    );
  }

  const modelConfig = GOVERNANCE.MODELS[providerId];

  // Rule 2: tier gate
  if (!tierAllowsFeature(req.userTier, req.feature)) {
    throw new GovernanceViolationError(
      "feature_not_available",
      `Tier ${req.userTier} cannot use feature ${req.feature}`,
    );
  }

  // Voice scope rules
  if (req.feature === "voice_input" && req.sessionType !== "mentor") {
    throw new GovernanceViolationError(
      "voice_scope_violation",
      "Whisper is mentor sessions only",
    );
  }
```

```
  if (
    req.feature === "voice_output" &&
    req.sessionType &&
    req.sessionType !== "mentor"
  ) {
    throw new GovernanceViolationError(
      "voice_scope_violation",
      "TTS is mentor responses only",
    );
  }

  const credentials = getCredentialsForFeature(req.feature);

  let systemPrompt = "";
  let requiresIntentDetectionFirst = false;
  let requiresUserPermission = false;

  if (req.feature === "mentor_message") {
    systemPrompt = buildMentorSystemPrompt(
      req.assertivenessLevel ?? 3,
      req.userData ?? {},
    );
  } else if (
    [
      "assistant_message",
      "roadmap_assistant_message",
      "journal_assistant_message",
      "image_upload",
    ].includes(req.feature)
  ) {
    requiresIntentDetectionFirst = true;
    requiresUserPermission = true;
    systemPrompt = buildAssistantSystemPrompt(
      req.assistantTone ?? "standard",
      req.shelf ?? {},
      req.preloadedData ?? {},
    );
  } else if (
    [
      "roadmap_generation",
      "roadmap_recalibration",
      "weekly_review",
      "monthly_breakdown",
      "daily_review",
    ].includes(req.feature)
  ) {
    systemPrompt = buildReviewSystemPrompt(
      req.periodType ?? req.feature,
      req.periodData ?? {},
    );
  } else if (req.feature === "intent_detection") {
    systemPrompt = INTENT_DETECTION_PROMPT;
  }

  return {
    provider: credentials.provider,
    credentials,
    systemPrompt,
    requiresIntentDetectionFirst,
    requiresUserPermission,
    backgroundJob: "backgroundJobsOnly" in modelConfig && !!modelConfig.backgroundJobsOnly,
  };
}
```